

Clase 016 — Python para seguridad: sockets y programación de red

Parte: **0 — Fundamentos y prerequisites** · Fuente: *Seitz & Arnold, Black Hat Python (2ª ed.)* 🕒
Duración estimada: **120 min** · Nivel: **Fundamentos**

Objetivo

Programar comunicaciones de red desde cero con la librería `socket`. Al terminar podrás construir clientes y servidores TCP/UDP, un escáner de puertos y un cliente/servidor tipo banner grabber, entendiendo cómo se traduce la teoría TCP/IP a código. Es la base de casi cualquier herramienta ofensiva de red.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

- 1. **Crear** clientes y servidores TCP con `socket`.
- 2. **Implementar** comunicación UDP.
- 3. **Escribir** un escáner de puertos con manejo de timeouts.
- 4. **Realizar** banner grabbing para identificar servicios.
- 5. **Aplicar** concurrencia (hilos) para acelerar tareas de red.

Temas

#	Tema	Por qué importa
1	API de sockets	<code>socket</code> , <code>connect</code> , <code>bind</code> , <code>listen</code>
2	Cliente TCP	Conectar y enviar/recibir datos
3	Servidor TCP	Aceptar conexiones
4	UDP	Envío sin conexión
5	Timeouts	Evitar bloqueos indefinidos
6	Escáner de puertos	Aplicación práctica directa
7	Banner grabbing	Identificar servicios y versiones
8	Concurrencia	Hilos para escanear rápido

Definiciones y características

- **Socket**: extremo de comunicación identificado por IP+puerto. Clave: `AF_INET` (IPv4) + `SOCK_STREAM` (TCP) o `SOCK_DGRAM` (UDP).

- **connect()**: inicia el handshake TCP hacia un destino. Clave: un `connect` exitoso implica puerto abierto (base del connect scan).
- **Timeout**: tiempo máximo de espera de una operación. Clave: sin él, un puerto filtrado cuelga el escáner.
- **Banner**: texto que muchos servicios envían al conectar (versión, software). Clave: revela información para fingerprinting.
- **Hilo (thread)**: flujo de ejecución concurrente. Clave: la red es I/O-bound, así que los hilos aceleran mucho el escaneo.
- **bind()/listen()**: preparan un servidor para aceptar conexiones. Clave: base de un handler/listener.

Herramientas y preparación

Solo necesitas Python 3 y la stdlib (`socket` , `threading` , `concurrent.futures`). Trabaja en tu laboratorio aislado con Kali y una VM víctima. Ten `nc` (netcat) a mano para levantar servicios de prueba y comparar comportamientos.

Laboratorio guiado

1. **Cliente TCP mínimo**. Conéctate a un servicio de la víctima y lee su banner:

```
python import socket s = socket.socket(socket.AF_INET, socket.SOCK_STREAM) s.settimeout(3)
s.connect(("10.10.10.6", 22)) print(s.recv(1024).decode(errors="replace")) s.close()
```

2. **Servidor de eco** en Kali (para entender el lado servidor):

```
python srv = socket.socket(); srv.bind(("0.0.0.0", 9000)); srv.listen(1) conn, addr =
srv.accept(); print("Conexión de", addr) conn.sendall(conn.recv(1024)); conn.close()
```

Pruébalo con `nc 10.10.10.5 9000`. 3. **UDP**. Envía un datagrama con `SOCK_DGRAM` y observa que no hay handshake. 4. **Escáner de puertos secuencial**. Recorre un rango con `connect_ex()` (que devuelve 0 si abre):

```
python for port in range(1, 1025): s = socket.socket(); s.settimeout(0.5) if
s.connect_ex(("10.10.10.6", port)) == 0: print(f"[+] {port} abierto") s.close()
```

5. **Banner grabbing**. Para cada puerto abierto, intenta `recv()` y guarda el banner.
6. **Acelerar con hilos**. Reescribe el escáner con `concurrent.futures.ThreadPoolExecutor` y compara tiempos.

 **Nota ética:** los escáneres y clientes se ejecutan **únicamente** contra tus VMs de laboratorio o sistemas autorizados. Escanear infraestructuras ajenas sin permiso es ilegal.

Ejercicios

1. Añade a tu escáner la resolución de nombre a IP con `socket.gethostbyname`.
2. Implementa un timeout configurable por argumento de línea de comandos.
3. Modifica el banner grabber para enviar `HEAD / HTTP/1.0\r\n\r\n` y capturar la respuesta de un servidor web.

4. Escribe un pequeño servidor TCP que registre cada conexión con su IP y hora.
5. Compara el tiempo de escaneo de 1024 puertos en versión secuencial vs. con 100 hilos.
6. Maneja correctamente `ConnectionRefusedError`, `socket.timeout` y `OSError`.

Reto verificable

Construye `pyscan.py`, un escáner de puertos TCP multihilo que reciba un objetivo y un rango de puertos, use timeouts, haga banner grabbing de los puertos abiertos y muestre un informe ordenado (puerto → estado → banner). Debe manejar errores de red y ser notablemente más rápido que la versión secuencial.

Criterio de aceptación: contra tu VM víctima detecta correctamente los puertos abiertos conocidos y captura al menos un banner (p. ej. SSH), completa el escaneo de 1024 puertos en una fracción del tiempo de la versión secuencial, y no se cuelga ante puertos filtrados. Comparable con la salida de `nmap`.

Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
El escáner se cuelga en algunos puertos	Falta <code>settimeout</code> . Fija un timeout corto por socket.
<code>OSError: Too many open files</code>	No cierras los sockets. Usa <code>with</code> o <code>close()</code> y limita hilos.
Todos los puertos parecen "abiertos" o "cerrados"	Confundes <code>connect_ex</code> (0 = abierto) con excepciones. Revisa el código de retorno.
Banner vacío en puertos abiertos	El servicio espera que hables tú primero (HTTP). Envía una petición antes de <code>recv</code> .
Datos ilegibles al imprimir	Son bytes. Decodifica con <code>errors="replace"</code> o trabaja en hex.

Preguntas frecuentes

? ¿Por qué mi escáner en Python es más lento que nmap? `nmap` está en C, usa raw sockets y técnicas avanzadas (SYN scan, temporización adaptativa). El objetivo aquí es entender el mecanismo, no superar a `nmap`.

? ¿connect scan o SYN scan en Python? Con `socket` haces connect scan (completa el handshake). Para SYN scan necesitas raw sockets/Scapy, que veremos en la Clase 017.

? ¿Hilos o asyncio? Para I/O de red ambos sirven. Los hilos son más sencillos de entender al principio; `asyncio` escala mejor a miles de conexiones.

? ¿Puedo escanear UDP con sockets? Sí, pero es poco fiable: la falta de respuesta no distingue "abierto" de "filtrado". Por eso el escaneo UDP es difícil incluso para `nmap`.

Referencias

- Seitz & Arnold, *Black Hat Python* (cap. de redes).
- Python `socket` — <https://docs.python.org/3/library/socket.html>
- Python `concurrent.futures` — <https://docs.python.org/3/library/concurrent.futures.html>

- Beej's Guide to Network Programming — <https://beej.us/guide/bgnet/>

Siguiente clase

Clase 017 - Python para seguridad: manipulacion de paquetes con Scapy